

Plataforma de expositores y credenciales

Gestiona las empresas que exponen en la feria y las credenciales del personal que opera sus stands. El administrador registra expositores y ve, stand por stand, cuántas credenciales hay asignadas y cuántas quedan; cada representante acredita a su gente —una a una o subiendo un Excel— dentro del cupo que le corresponde por metraje.

EL EJE DEL DISEÑO

El enunciado pide que se puedan crear ferias futuras **sin que los datos se mezclen**. Eso no es una nota al pie: es lo que ordena el modelo. Toda tabla operativa lleva `event_id`, ningún repositorio se construye sin él, y ese `event_id` sale siempre del token, nunca de la petición.

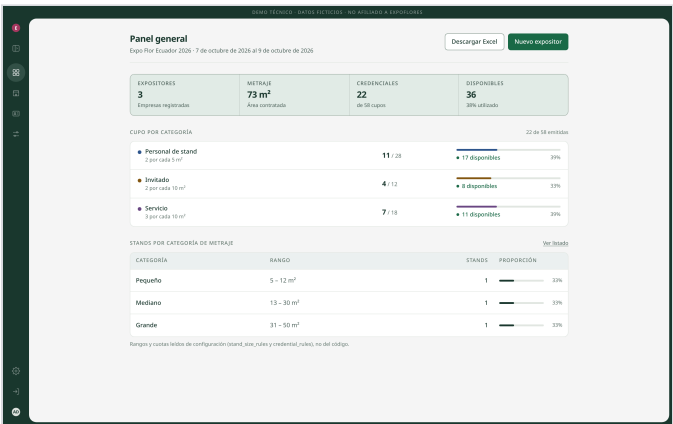
La segunda columna vertebral es la **parametrización**: rangos de metraje, cuotas por categoría y hasta el criterio de redondeo viven en tablas. En el repositorio no hay ningún `if m2 > 12`.

ALCANCE ENTREGADO

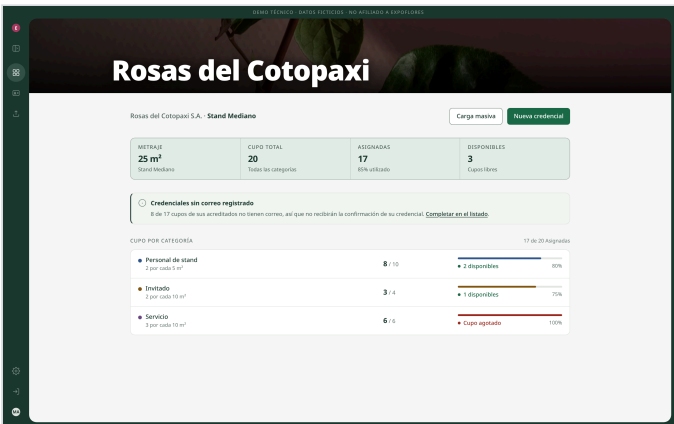
Administrador CRUD de expositores con alta atómica de la cuenta del representante, panel de asignadas vs. disponibles, reporte en Excel, reenvío del enlace de acceso.

Representante Panel de cupo por categoría, alta y baja de credenciales, carga masiva desde Excel con vista previa.

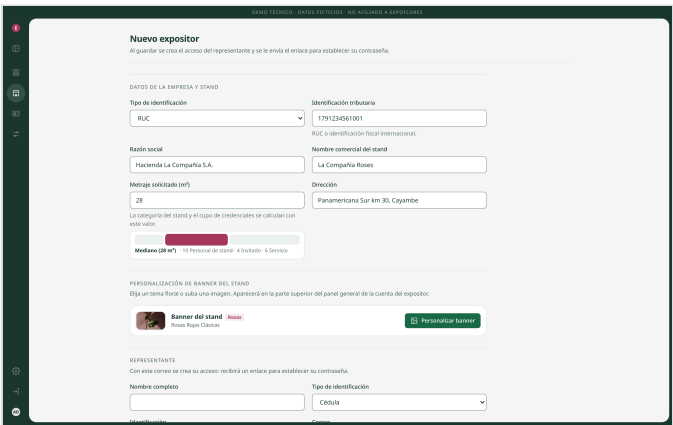
Puntos extra Los tres: correos transaccionales, carga masiva leída en el navegador y reglas parametrizadas en base de datos.



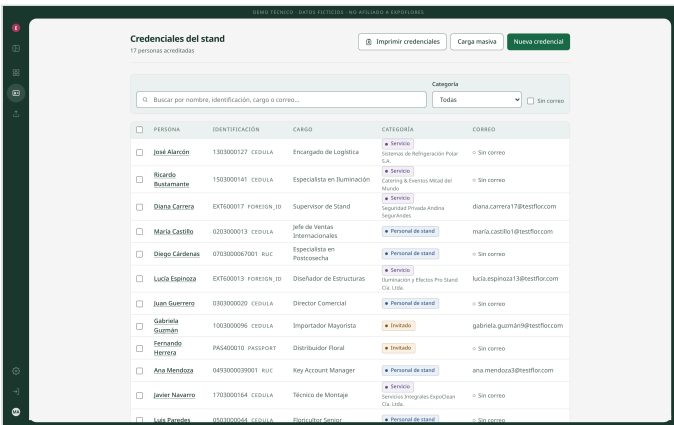
Panel del administrador: cupo, asignadas y disponibles por stand, con la categoría derivada del metraje.



Panel del representante: cupo por categoría del propio stand, calculado en cada lectura, nunca leído de una columna.



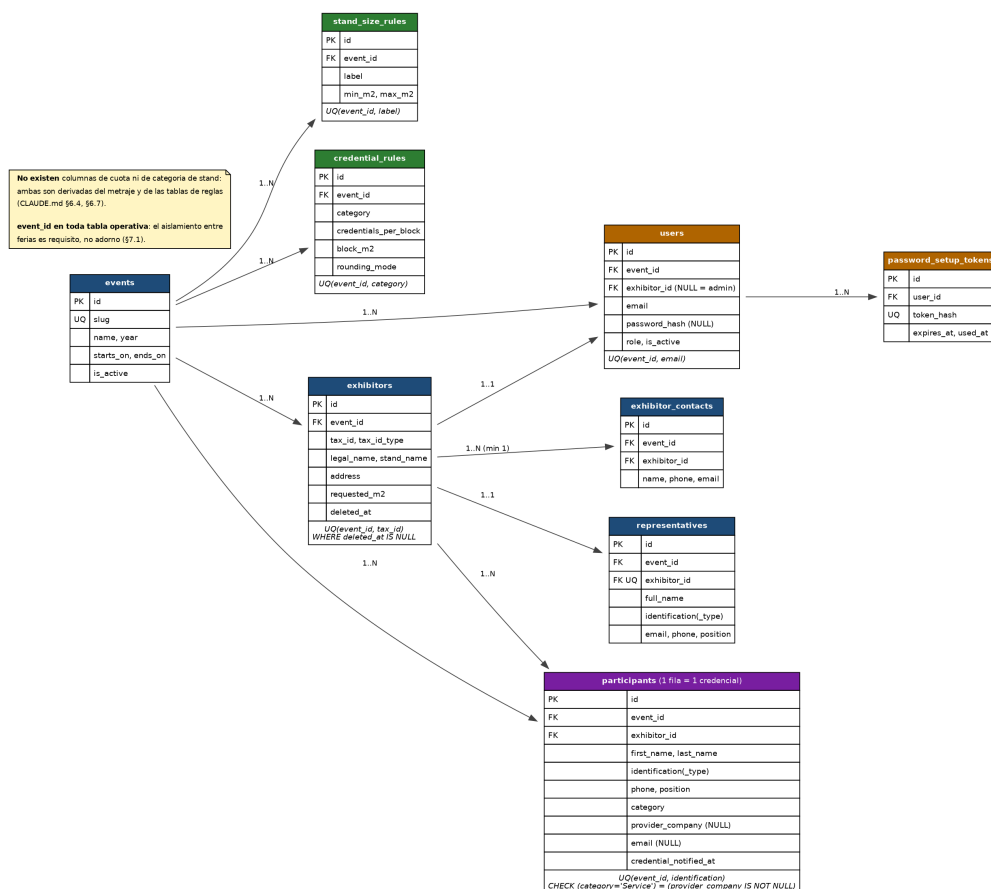
Alta de expositor: crea en una sola transacción la empresa, su representante, los contactos, la cuenta de acceso y el token de contraseña.



Credenciales del stand. Eliminar una libera cupo y libera la identificación para otro stand de la feria.

Modelo de datos

Doce tablas, dos de ellas de configuración. Ninguna cifra de negocio es una columna calculada ni un valor congelado.



Por qué el scope por evento está en todas partes

- **Toda tabla operativa lleva event_id**, incluso donde sería deducible por la relación padre. Deducirlo obliga a un JOIN para no filtrar datos; tenerlo permite que la condición esté siempre en el WHERE.
- **Las claves únicas son compuestas, nunca simples.** UNIQUE(event_id, identification) implementa la validación crítica y a la vez permite que la misma persona vuelva en la edición siguiente con otra empresa. Una restricción global habría bloqueado registros legítimos.
- **El repositorio no se puede construir sin event_id.** No es comodidad: es la barrera que impide que llegue a existir un método de listado sin scope, que es por donde se cuegan las fugas entre ferias y los IDOR.
- **No hay columna de cuota ni de categoría de stand.** Ambas se derivan del metraje y de las reglas vigentes en cada lectura. Una columna congelada se desincroniza en cuanto alguien corrige un metraje.
- **Borrado lógico solo en expositores.** Borrar en duro una empresa con credenciales emitidas es destructivo. Los participantes sí se borran de verdad: borrar una credencial libera cupo y libera la identificación.
- **password_hash es nutable** hasta que el representante establece su clave con el token de un solo uso. Un usuario sin hash no puede autenticarse: el estado "cuenta creada, aún no activada" es explícito.

Parametrización

El enunciado dice que se evaluará si los rangos son modificables o están quemados en el código. Están en dos tablas, y el motor de reglas las recibe como parámetro.

STAND_SIZE_RULES · CLASIFICACIÓN

CATEGORÍA	MIN_M2	MAX_M2
Pequeño	5	12
Mediano	13	30
Grande	31	50

Es **informativa y de validación de rango**: no interviene en el cálculo de la cuota. Un metraje fuera de [5, 50] se rechaza con STAND_SIZE_OUT_OF_RANGE en vez de clasificarse por aproximación.

CREDENTIAL_RULES · CUOTA

CATEGORÍA	PER_BLOCK	BLOCK_M2	ROUNDING
Exhibitor	2	5	floor
Guest	2	10	floor
Service	3	10	floor

```
cuota = credentials_per_block * redondeo(m² / block_m2)

25 m² → Exhibitor 10 · Guest 4 · Service 6
8 m² → Exhibitor 2 · Guest 0 · Service 0
```

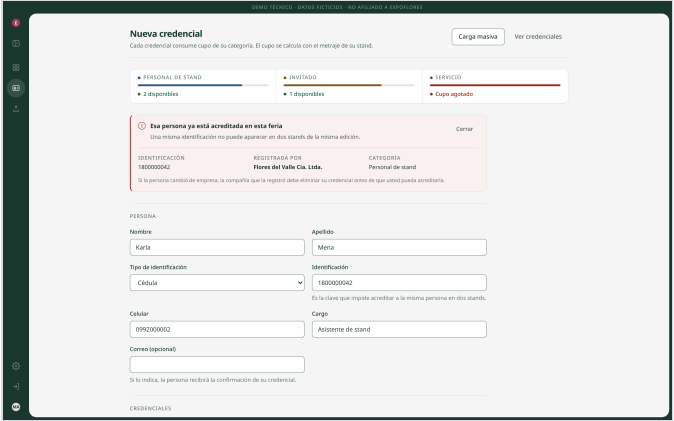
CAMBIAR UNA REGLA SIN TOCAR CÓDIGO NI REDESPLEGAR

```
UPDATE stand_size_rules SET max_m2 = 80 WHERE event_id = 1 AND label = 'Grande';
UPDATE credential_rules SET rounding_mode = 'ceil' WHERE event_id = 1 AND category = 'Guest';
```

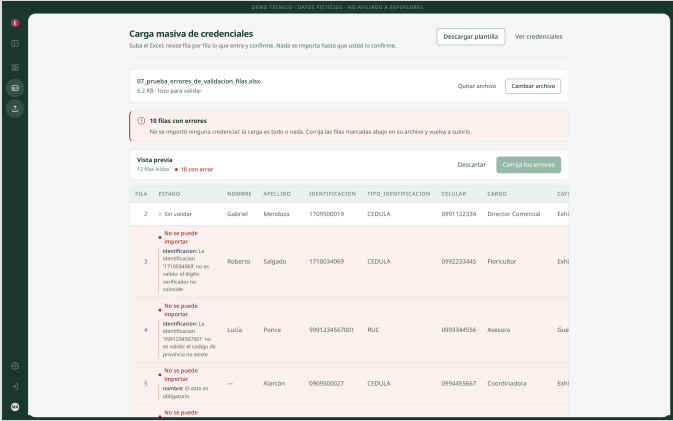
El motor de reglas es un módulo puro —sin SQLAlchemy, sin FastAPI— que recibe las filas como parámetro y no las cachea: el cambio surte efecto en la siguiente petición. El test test_changing_range_changes_classification modifica un rango por SQL y comprueba que la clasificación cambia sola.



La pantalla de reglas lee esas mismas dos tablas en cada petición e incluye un simulador que deriva categoría y cuota de cualquier metraje con la misma función que usa el alta real.



Validación crítica: una identificación no puede estar en dos stands de la misma feria. El bloque es persistente y nombra a la empresa que ya la registró.



Carga masiva: el navegador lee el Excel, el servidor lo valida fila por fila y nada se importa mientras haya un error.

Ambigüedades detectadas

Ocho cuestiones que el enunciado deja abiertas y que no se pueden implementar sin decidir. Criterio general: **la opción que falla en voz alta gana a la que adivina en silencio.**

AMBIGÜEDAD	DECISIÓN	POR QUÉ
1. Redondeo. 13 m² con "2 por cada 5 m²": ¿4 o 6?	floor, y el criterio es una columna (rounding_mode).	Es la lectura literal de "por cada N m²" y la conservadora para el organizador. Como es un dato, otra feria decide distinto con un UPDATE.
2. Metrajes fuera de rango. Los rangos cubren 5–50 m². ¿Y 60?	Rechazar con STAND_SIZE_OUT_OF_RANGE.	Clasificar por aproximación crea datos erróneos silenciosos en la columna que alimenta el cupo de todo el stand. Un error explícito es recuperable.
3. ¿El representante consume credencial?	No. Si opera el stand, se registra además como participante.	Separa la identidad de acceso de la credencial física. Un descuento invisible de cupo es imposible de explicar seis meses después.
4. ¿La cuota se congela al crear el expositor?	Cálculo derivado siempre. Si el metraje nuevo deja la cuota bajo lo asignado, se bloquea el cambio.	Una columna congelada se desincroniza en cuanto alguien corrige un metraje. El bloqueo convierte el único caso peligroso en un error legible.
5. ¿Cómo recibe la contraseña el usuario auto-creado?	No se envía ninguna: token de un solo uso, 72 h, guardado hasheado.	El correo es un canal persistente y sin cifrar en reposo. Una contraseña enviada por correo sigue ahí dentro de dos años.
6. ¿La unicidad de identificación es global o por evento?	Por evento: UNIQUE(event_id, identification).	Las ediciones son independientes y la gente cambia de empleador. Global bloquearía registros legítimos y rompería el aislamiento entre ferias.
7. ¿La categoría de stand influye en la cuota? DETECTADA	No. Es informativa y de validación de rango.	Las cuotas están expresadas en m², no por categoría. Cruzarlas haría que ampliar un rango cambiara silenciosamente cuotas de stands que nadie tocó.
8. Hay que notificar al participante, pero el correo no está entre sus datos requeridos. CONTRADICCIÓN	email opcional: si viene se notifica, si no, el alta no falla.	Exigirlo contradiría la lista de datos del propio enunciado y bloquearía altas legítimas. Notificar al representante se descartó: el enunciado nombra al participante.

DÓNDE VIVE CADA DECISIÓN

1, 2 y 7 — el motor de reglas.

app/domain/rules.py es un módulo puro: recibe las filas de configuración como parámetro y se prueba sin base de datos, incluidos los bordes 12, 13, 30 y 31 m² y los tres modos de redondeo.

4, 6 y 8 — el modelo.

Sin columna de cuota, UNIQUE(event_id, identification) y email nutable. Lo que el esquema no permite no hace falta recordarlo en el código.

3 y 5 — el alta de expositor.

Una transacción crea empresa, representante, contactos, usuario sin contraseña y token; el correo con el enlace sale después del COMMIT.

Trade-offs y producción

Lo que sí se cuidó

- **El cupo se verifica dentro de la transacción, con SELECT ... FOR UPDATE** sobre la fila del expositor. Sin el bloqueo, dos altas simultáneas sobre la última credencial pasan ambas la verificación. La carga masiva reutiliza ese servicio y comprueba el lote completo de una vez.
- **Un recurso ajeno responde 404, no 403.** No existe endpoint del representante que acepte un id de empresa: el scope sale del token.
- **Duplicado validado en dos capas** —servicio y constraint — que producen exactamente el mismo error estructurado. El cliente no puede distinguir cuál disparó.
- **Los correos salen después del COMMIT.** Un SMTP caído registra el fallo y no tumba el alta; hay un test que lo demuestra.
- **Cédula y RUC con el algoritmo real** (módulo 10 y módulo 11 según el tercer dígito). Es la clave de deduplicación de todo el sistema.

Lo que se dejó fuera a propósito

Robustez es hacer bien las piezas necesarias, no agregar piezas. No hay colas, ni caché distribuida, ni microservicios, ni un design system propio: nada de eso resolvía un problema que este MVP tenga.

Qué cambiaría en producción

- **Refresh tokens y revocación.** Hoy el JWT dura 60 minutos y no se puede invalidar antes de tiempo.
- **Envío de correo fuera del ciclo de la petición.** Ya está aislado tras una función y no bloquea la transacción, pero un reintento con backoff necesita un worker.
- **Rate limiting con almacenamiento compartido.** El contador vive en memoria del proceso; con varias réplicas hay que centralizarlo.
- **Row-level security en Postgres** como segunda barrera del scope por evento, además de la del repositorio.
- **Observabilidad.** Hay logs JSON con `request_id`; faltan métricas y trazas.
- **Selector multi-evento para el administrador.** El esquema ya lo soporta; la interfaz opera sobre el evento activo.

PRUEBAS

Cada requisito evaluable tiene un test que lo demuestra, con su tabla de trazabilidad en el README. El motor de reglas se prueba sin base de datos.

PUESTA EN MARCHA

`docker compose up -d --build` levanta base, API y frontend, aplica las migraciones y siembra datos ficticios. Credenciales de demo en el README y en la propia pantalla de login.

CALIDAD

`ruff` y `mypy --strict` limpios en el backend; TypeScript estricto sin `any` en el frontend, con los tipos de la API generados desde el esquema OpenAPI.